

Definición del stack tecnológico

¿Qué es un stack tecnológico?

Un *stack* tecnológico es un conjunto de tecnologías que se combinan para construir y ejecutar una aplicación. Esto, incluye al menos:

- Lenguaje(s) de programación
- Frameworks, paquetes o librerías
- Motor de bases de datos
- Entorno o plataforma de despliegue y ejecución

Algunos de los más frecuentemente utilizados son:

- LAMP: Linux - Apache - MySQL - PHP
- MERN: MongoDB - Express - React - NodeJS
- MEAN: MongoDB - Express - Angular - NodeJS
- PERN: PostgreSQL - Express - React - NodeJS

Existe tanta variación como combinaciones posibles de distintas tecnologías. Más allá de los nombres, lo importante es entender que es un **conjunto de componentes**, donde **cada uno cumple un rol** o se encarga de una responsabilidad particular.

Principio fundamental a la hora de elegir un stack

Siempre que se cuente con libertad de elección y sea viable para el proyecto que se quiere desarrollar, **el mejor stack es el que ya se domina o se conoce**.

Costo de aprender una nueva tecnología

Aprender a usar una nueva herramienta y llevar su uso a un buen nivel de dominio lleva tiempo. Si se cuenta con un plazo de tiempo determinado para resolver un problema de negocio, se genera una competencia por ese período entre el tiempo destinado al aprendizaje y el destinado al desarrollo de la solución. En un contexto académico con fechas de entregas pactadas o trabajos con clientes, el costo de aprendizaje no siempre se puede afrontar.

Al trabajar con tecnologías conocidas:

- Se escribe y revisa código más rápidamente.
- Se reconocen con mayor frecuencia y rapidez los errores comunes.
- Se conocen las limitaciones desde el inicio y se puede diseñar la solución en consecuencia a ello.

Costo de cambio en etapa de desarrollo

Uno de los escenarios más costosos de un proyecto es descubrir, cuando ya está avanzado, que la tecnología elegida no se comporta como se esperaba. Migrar de un modelo de bases de datos a otro, cambiar de framework o reescribir un backend en otro lenguaje son

situaciones que ocurren y que casi siempre tienen su origen en haber elegido algo sin conocimiento en profundidad de cómo la tecnología funciona realmente.

Condiciones de exploración

No siempre un stack conocido es la solución. Puede suceder que no se ajuste al tipo de proyecto que tenemos que llevar a cabo, o que al conocer las limitaciones sepamos que puede funcionar en el corto plazo pero si la solución debe escalar de manera exponencial. En general, buenas situaciones para explorar nuevas tecnologías presentan algunas de estas características:

- Se trata de un proyecto personal o académico con condiciones flexibles, y **el objetivo es aprender**.
- La nueva tecnología **resuelve un problema particular** que la actual o conocida no puede resolver fácilmente.
- Los tiempos son lo suficientemente holgados y **admiten la curva de aprendizaje**.
- Se cuenta con algún integrante del equipo que ya domina la tecnología y puede guiar al resto.

Criterios técnicos para la elección del stack

Cuando el stack conocido no alcanza, o cuando se está evaluando entre dos tecnologías que ya se manejan, entran en juego ciertos criterios técnicos que aseguran viabilidad y mayores probabilidades de éxito técnico.

Naturaleza del problema

No cualquier tecnología se adapta igual a todos los problemas. Pueden considerarse los siguiente ejemplos:

- Sistemas de alta concurrencia (muchos usuarios simultáneos, tiempo real): Node.js con su modelo de I/O no bloqueante tiene ventajas claras. Go es otra opción popular en este espacio.
- Procesamiento de datos, machine learning, scripts de análisis: Python domina este dominio por su ecosistema (Pandas, Scikit-learn, PyTorch, etc.).
- Aplicaciones web tradicionales con lógica de negocio compleja: frameworks maduros como Django (Python), Laravel (PHP), Spring Boot (Java) o ASP.NET Core (C#) son sólidos.
- Aplicaciones móviles nativas: Swift (apps de iOS), Kotlin (apps de Android), o React Native / Flutter para multiplataforma.

Escala esperada

Un sistema que vaya a ser usado por 50 usuarios internos de una organización tiene requerimientos muy distintos a uno que tenga miles de usuarios concurrentes. La elección de tecnologías debe considerar el **escenario real analizado**, y no uno imaginado, en el que el sistema va a funcionar.

La **sobreingeniería** es tan problemática como **subingeniería** de un sistema. Construir una arquitectura de microservicios con Kubernetes para gestionar el inventario de un negocio familiar no es una buena decisión técnica, aunque sea tecnológicamente impresionante.

Madurez de la tecnología

Una nueva tecnología que está siendo difundida por redes sociales e influencers como la que va a reemplazar a todas las existentes, puede ser un gran riesgo para un proyecto. Si el ecosistema no es lo suficientemente maduro pueden encontrarse bugs, errores o limitaciones no documentadas que no tengan forma clara de sobrellevar. Por ello, algunos aspectos importantes a tener en cuenta pueden ser los siguientes:

- Nivel de **actividad** de la comunidad en foros o medios técnicos.
- Frecuencia de actualizaciones y **mantenimiento**.
- Existencia de **librerías** o paquetes para mi caso de uso particular.
- **Casos exitosos** de su uso en producción para problemas similares al tratado.

Restricciones del entorno de despliegue y recursos existentes

Muchas veces, la tecnología que usamos está limitada por los recursos técnicos pre-existentes o disponibles. Por ejemplo:

- Si el servicio de hosting disponible soporta solo un tipo de lenguaje y no cambiarlo no es la primera opción, deberíamos adaptarnos a ello de la mejor manera posible, siempre y cuando sea viable.
- Si el sistema debe construirse sobre fuentes de datos, como una base de datos SQL, puede tener sentido usar la tecnología que mejor se integre con ella en lugar de crear una nueva base de datos.

Elecciones de lenguajes

Lado del cliente

En este caso, no contamos con tanta variedad como en los lenguajes para servicios backend. Generalmente, el **tipo de aplicación y plataforma** la que esté destinada va a ser quien dicte el lenguaje a utilizar:

- Aplicaciones Web: Los navegadores solo interpretan JavaScript de forma nativa, así que es el lenguaje predilecto, con o sin frameworks, para este tipo, acompañado de tecnologías como HTML y CSS.
- Aplicaciones de escritorio para sistemas Windows: Casi con seguridad se usarían las plataformas de desarrollo .NET o .NET Framework, implementando frameworks como MAUI, WPF, o incluso WinForms, utilizando C# o VB.NET. También existen otras alternativas con JavaScript a través de frameworks como ElectronJS.
- Aplicaciones móviles: De acuerdo a la plataforma, pueden usarse Swift (iOS) o Kotlin (Android), o React Native (multiplataforma).

Lado del servidor

En este caso, hay muchísimas alternativas que se adaptan a distintos escenarios según lo que necesite el problema en cuestión:

Lenguaje	Framework	Escenario ideal
JavaScript/TypeScript	Express, NestJS	Cuando se requiere unificación de lenguaje
Python	Django, Flask	Hay componentes de datos o machine learning
PHP	Slim, Laravel	Proyectos web tradicionales y transaccionales
Java	Spring Boot	Entornos empresariales, seguridad de tipado
Golang	Estándar, Gin	Alta performance en concurrencia, servicios de infraestructura
C#	ASP.NET Core	Entornos empresariales con servicios de Microsoft y requieran integración

Elección del tipo de base de datos

Bases de datos relacionales

Organizan los datos en tablas con **esquemas definidos y estrictos**. Usan SQL como lenguaje de consulta. Son la opción más adecuada cuando:

- Los datos tienen estructura bien definida y relativamente estable.
- Las relaciones entre entidades son importantes (claves foráneas, joins).
- Se necesita integridad transaccional (ACID).

Bases de datos no relacionales

Almacenan datos en formatos distintos al relacional, diseñados para **propósitos específicos**: documentos JSON, grafos, clave-valor, columnas anchas. Son más adecuadas cuando:

- Los datos no tienen una estructura fija o cambia frecuentemente.
- Se necesita escalar horizontalmente con grandes volúmenes.
- El modelo de datos es naturalmente jerárquico o de documentos.
- El diseño de la BD se ajusta específicamente a nuestro caso de uso

Tecnología de despliegue

La forma en que una aplicación se despliega y ejecuta en un servidor también es una decisión técnica.

Servidor tradicional (bare metal, VMs o VPS)

La aplicación corre directamente sobre el sistema operativo del servidor. Es el modelo **más simple** conceptualmente: se instalan las dependencias, se configura el entorno y se ejecuta el proceso. Requiere gestión manual del servidor pero da control total sobre el entorno.

Contenedores (Docker)

Permiten empaquetar la aplicación junto con todas sus dependencias en una **unidad aislada y reproducible**. Resuelve el problema clásico de diferencias entre entornos de desarrollo y producción. Es posible orquestar múltiples contenedores (por ejemplo, frontend + backend + base de datos) con un solo archivo de configuración, lo que lo hace especialmente útil para trabajo en equipo y pruebas.

Serverless

El código se despliega como funciones individuales que se ejecutan bajo demanda. No hay un servidor que gestionar. Es adecuado para **backends sin estado o APIs simples**, pero tiene limitaciones para aplicaciones con procesos largos o conexiones persistentes.

PaaS (Plataforma como servicio)

Abstrae la infraestructura subyacente. Solo se sube el código y la plataforma se encarga del resto. **Reduce la complejidad operativa a costa de menor control.**

Justificación del stack elegido

Elegir un stack es solo la mitad de la tarea. La otra mitad es justificarlo. Una justificación técnica sólida responde al menos estas preguntas:

- ¿Por qué este lenguaje y framework para el frontend?
- ¿Por qué este lenguaje y framework para el backend? ¿Qué problema resuelve mejor que las alternativas?
- ¿Por qué este gestor de base de datos? ¿Es SQL o NoSQL?
- ¿Por qué esta plataforma de despliegue? ¿Qué restricciones técnicas o económicas influyeron?
- ¿El equipo (o el estudiante) tiene experiencia previa con estas tecnologías?

La justificación no necesita ser extensa, pero sí debe ser honesta. "Elegí React porque lo conozco y hay muchos recursos de aprendizaje" es una justificación válida. "Elegí React porque es el framework más utilizado" no lo es.